AI 팩토리 구축 이론

루프 · 하네스 · 그래프

> 반복이 성립하는 조건 · 루프와 하네스
> 그래프의 상태 · 간선 · 분기, 그리고 루프와의 비교

도구 및 코드보다 **아키텍처 구조와 개념**에 집중한다.

2026년 타임라인

- **6월 7일 — Addy Osmani**
  - *Loop Engineering* 발표 : 사람이 매번 프롬프트를 입력하는 대신, 에이전트를 반복 실행하고 검증하는 시스템을 설계하자고 제안
  - 핵심 정의: **“Loop engineering is replacing yourself as the person who prompts the agent.”**
- **7월 4일 — Josh C. Simmons**
  - *We Are Entering the Graph Engineering Phase* 발표
  - 명시적 노드, 타입이 있는 엣지, 체크포인트 상태를 가진 에이전트 시스템을 “그래프 엔지니어링”이라고 정의
- **7월 18일 — Peter Steinberger**
  - “우리 아직 루프 얘기 중인가, 벌써 그래프로 넘어갔나?”
  - 새로운 기술을 발표한 것이 아니라 빠른 용어 교체에 대한,,,
- **같은 날 — Hamel Husain**
  - *Loop Engineering Is Dead. Enter Graph Engineering*
  - 기술 논문이나 선언문이 아니라 농담에 가까운 반응

- **7월 18~20일**
  - 해설, 로드맵, 강의, 툴 스택 콘텐츠가 빠르게 파생,,,
  - 기존 기술이 새로운 필수 역량처럼 포장되기 시작,,,
- **7월 22일 — LangChain**
  - 그래프 기반 에이전트 설계는 새 개념이 아니라는 공식 설명 발표
  - LangGraph는 이미 수년간 같은 구조를 구현해 왔음

그래프란, 이미 존재하던 설계 방식 중 하나.
상태 머신, 워크플로 그래프, DAG, 재시도와 체크포인트는 기존 소프트웨어·분산 시스템에서 오랫동안 사용된 개념.

따라서 정확한 결론은 다음과 같습니다.

- **새로운 것:** “그래프 엔지니어링”이라는 명칭의 대중화
- **새롭지 않은 것:** 노드·엣지·상태·조건 분기로 시스템을 제어하는 구조
- **최근 중요해진 것:** 각 노드 안에 단순 LLM 호출이 아니라 완전한 에이전트 실행을 넣을 수 있게 된 점

핵심 : 필요성을 못 느낀다면, 안 쓰는게 정답

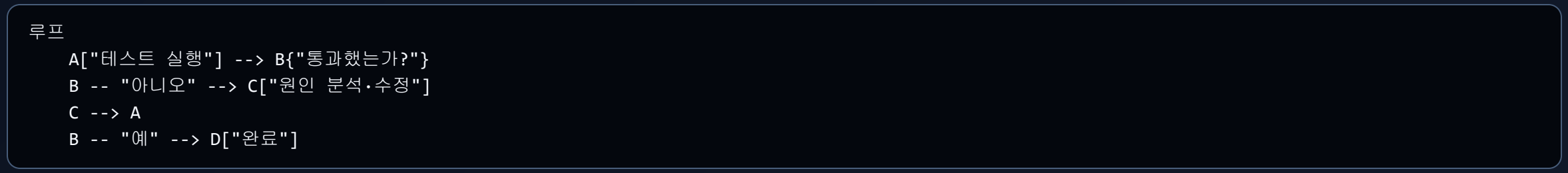
루프와 그래프의 관계

루프는 죽지 않았다 — 그래프 안으로 들어갔다

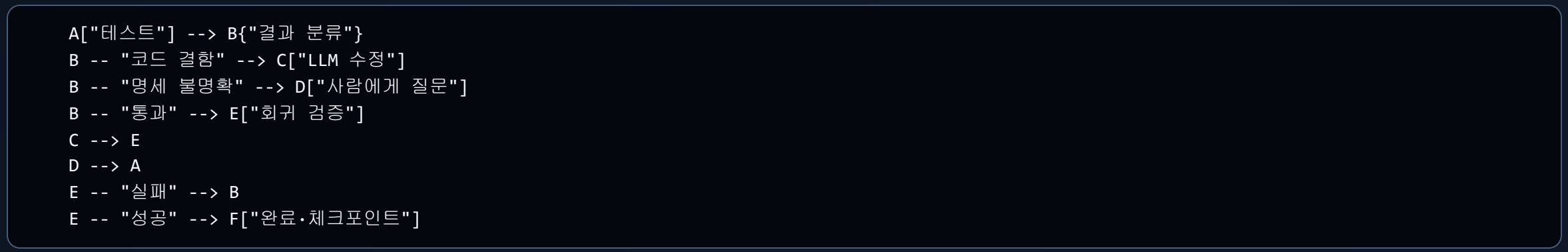
화면에 넣을 핵심 비교

구분	루프 엔지니어링	그래프 엔지니어링
핵심 질문	“언제까지 반복할 것인가?”	“다음에 어디로 이동할 것인가?”
기본 구조	작업 → 검증 → 실패 시 반복	여러 노드와 조건부 전이
제어 흐름	하나의 반복 흐름에 집중	분기·병렬·합류·승인·재시도 명시
상태	대화 기록·상태 파일 중심	스키마가 있는 상태 객체와 체크포인트
실패 처리	같은 루프 안에서 재시도	실패 유형별 복구 경로 분리
적합한 작업	경로가 단순하고 종료 조건이 명확한 작업	경로가 여러 개이고 단계별 통제가 필요한 작업
관계	단순한 순환 그래프	내부에 여러 루프를 포함할 수 있음

핵심 구조



위에도 **방향성과 순환을 가진 그래프**다. LangChain의 표현대로 **루프는 단순한 순환 그래프**.



그래프 엔지니어링은 여기에 복구 경로와 승인 지점 등을 명시적으로 추가.
그래프는 루프의 반대 개념이 아니라 루프를 더 큰 제어 구조 안에 배치한 것.

> **루프 엔지니어링은 한 작업을 안정적으로 반복시키는 기술이고, 그래프 엔지니어링은 여러 루프와 결정 지점을 하나의 시스템으로 조직하는 기술이다.**

그래프는 언제 필요한가?

모든 루프를 그래프로 바꿔야 하는 것은 아니다

루프만으로 충분한 경우

* 실행 경로가 사실상 하나다.
* 실패하면 같은 수정 단계로 돌아가면 된다.
* 작업을 병렬로 나눌 필요가 없다.
* 중간에 사람의 승인을 받을 필요가 없다.
* 하나의 상태 파일과 종료 조건으로 통제할 수 있다.

코드 작성 → 테스트 → 실패하면 수정 → 전부 통과하면 종료

그래프로 확장해야 하는 신호

* 실패 원인마다 복구 방법이 다르다.
* 독립적인 작업을 병렬 실행해야 한다.
* 작성자와 검증자를 분리해야 한다.
* 특정 단계에서 사람의 승인이 필요하다.
* 실행을 중단했다가 체크포인트부터 재개해야 한다.
* 단계별 비용·시간·반복 횟수를 제한해야 한다.
* 전체 결과뿐 아니라 어떤 경로로 결과가 만들어졌는지 평가해야 한다.

> **루프는 반복을 자동화하고, 그래프는 반복들 사이의 관계와 책임을 아래 구조도 기반으로 설계한다.**

구성요소	실습에서의 역할
State	테스트 결과, 실패 로그, 반복 횟수, 수정 파일, 실행 ID
Node	테스트, 원인 분석, 코드 수정, 회귀 검증, 증거 저장
Edge	PASS/FAIL에 따른 다음 단계
Checkpoint	각 실행 결과와 소스 스냅샷 저장
LLM	실패 원인 분석과 제한된 코드 수정
결정론적 코드	테스트 실행, 해시 검증, 반복 제한, 완료 판정
Human-in-the-loop	명세가 불명확하거나 권한이 필요한 경우 개입

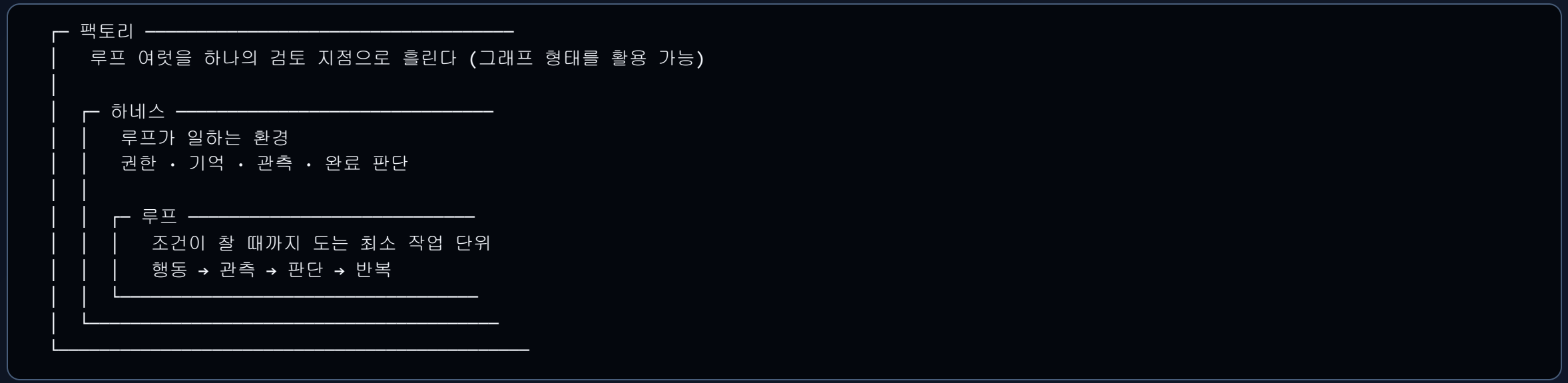
> **즉,그래프 엔지니어링은 비결정적인 LLM 판단과 결정론적인 시스템 제어(코드)를 노드와 엣지로 분리하는 것이다.**

LangChain 공식 설명

모든 작업을 그래프로 강제하는 것은 잘못이다. 범용 딥리서치처럼 실행 경로를 미리 규정하기 어려운 작업은 유연한 에이전트 루프가 더 적합하다. 반면 테스트·승인·복구 경로처럼 사전에 알고 있는 구조는 그래프로 명시하는 편이 안전.

“루프에서 그래프로의 교체”가 아니라 “단일 루프에서 필요하다면, 명시적인 상태 기반 워크플로우의 확장”

루프 · 하네스 · 팩토리



팩토리 : Scale up 조직도

출처: Addy Osmani, "Software Factories, Light and Dark" · addyo.substack.com

루프 — 조건이 찰 때까지 도는 최소 작업 단위

> 의도(intent) 기반의 목적(goal)을 조건으로 구축하고, 조건을 전부 달성할 때까지

Single Turn

- 시킨다 → 받는다 → 내가 읽는다
- 됐는지는 사람이 판단한다

Loop

- 시킨다 → 기계가 검사한다
- 조건이 차면 스스로 멈춘다

멈춤 조건이 곧 루프를 만든다.

한 바퀴 (1 Loop) — AORR

> AORR — 행동(Act) · 관측(Observe) · 판단(Reason) · 반복(Repeat)

```
Act       한 바퀴에 작업 하나만 손댄다
 ▼
Observe   검사기를 돌리고 종료 코드를 읽는다
 ▼
Reason    실패 유형을 고른다
 ▼
Repeat    그 유형에 정해진 다음 행동을 한다
 └──────▶  다시 Act 로
```

Observe 와 Reason 설계가 중요하다.

판정표 예시

검사 항목	목표	실제	판정
필수 항목 개수	9	8	**부족**
설명 분량	120자	214자	통과
링크가 살아 있는가	11	0	**부족**

행동(Act) : 서치엔진으로 정보 수집
관찰(Observe) : **목표 미달성 2건**
실패 유형 분석(Reason) : **정보 불충분**
다음 행동(Repeat) : 부족으로 표시된 칸만 다시 서치

0. 목적은 무엇인가

ex) 여행 추천을 진행하는 웹 페이지를 만들고 싶다.

1. 시작 상태 · 2. 관측 표면

시작 상태

루프가 **어떤 정보를 갖고, 무엇을 기준으로** 돌아야하는가.

ex) 내가 국가와 여행 일수 정도는 정보로 제공하자.

- 입력 · 제약 · 목표를 처음에 고정한다.
- 흔들리면, 재현성이 부숴진다.

관측 표면

목적을 위한 세부 달성 사항들

ex) 여행 추천 웹에는 총 3 개의 기능으로 구축하도록 하자.

에이전트가 **무엇을 보고** 달성/미달성을 판단하는가-를 설계

- 입력한 재료들이 어떻게 정렬되고, 정량화되는가.
- 볼 수 없으면 판정도 없다

관측 표면이 곧 목표 달성을 검증하는 TEST가 된다. 대충 설계하면, 대충 만들어도 통과한다.

3. 충족 기준 — "목표 달성"을 숫자로 바꾼다

숫자로 바꾸지 않으면, 종료 기준은 모호해진다. 그런데 숫자로 바꾸면, **함정**이 존재한다.

걸어 둔 기준	실제로 나온 것	왜
풍부한 설명을 위해 최소 130자 이상 작성	130자에서 끊김	하한이 목표가 된다
설명을 30줄 이상	20줄이 동일 내용 복붙	줄 수가 목표가 된다

검사은 통과한다. 그러나, 유효하지 않다. 분량 · 중복 · 유효성을 **함께** 걸어야한다.

4. 행동 경계 · 5. 반복 규칙 — 범위와 다음 진행 결정

④ 행동 경계

한 바퀴가 **무엇을 손대는가**.

- 한 바퀴에 작업 하나만
- 못 건드릴 것은 아예 잠근다
- 범위가 넓으면 원인을 못 찾는다

⑥ 반복 규칙

실패하면 **다음에 무엇을 하는가**.

- 실패 유형 7종 → 정해진 행동 하나
- 재시도 · 재조사 · 명세 수정 · 사람 · 중지

실패했을 때 할 일을 미리 정해 두지 않으면, 같은 실패를 반복한다.

루프 설계 순서

```
0  목적 설계     만들려고 하는게 무엇인가 & 해결하려고 하는게 무엇인가
1  시작 상태     무엇을 재료로 도는가
2  관측 표면     목적은 어떻게 세부 달성 기능으로 분해되는가, 무엇을 보고 판단하는가
▼
3  충족 기준     몇 개면, 어디까지면 됐다고 할 것인가
▼
4  행동 경계     한 루프가 무엇을 손대는가
5  판정 독립     누가 채점하는가
▼
6  반복 규칙     실패하면 무엇을 할 것인가
▼
7  종료 선언     무엇이 되면 끝인가
```

> 무엇을 모을지 모르면 **몇 개가 충분한지**도 정할 수 없다. 누가 채점하는지 모르면 **실패했는지**도 알 수 없다.

판정 독립 — 달성/미달성 판정은 고정되어야 한다

```
계약      분량 · 개수 · 상한 — 숫자는 여기에만 있다
  |
  ▼
검사기    계약에서 숫자를 읽어 판정한다 · 잠겨 있다
  |
  ▼
루프      결과물만 고친다

막는 길 두 개
  ✗ 루프가 검사기를 고친다
  ✗ 루프가 계약의 숫자를 낮춘다
```

검사기는 숫자를 갖지 않고, 계약에서 읽어온다.

종료 선언 — 끝을 선언하지 않으면 끝나지 않는다.

> "잘 된 것 같습니다"는 종료 조건이 아니다. 끝은 **종료 코드 3가지** 중 하나로 선언한다.

종료 코드	뜻	그래서 무엇을 하나
`0`	통과	끝낸다
`1`	깨졌다	사람이 고친다. 더 돌려도 안 된다
`2`	아직 덜 됐다	정상이다. 계속 돈다

> 좋은 루프는 빨리 도는 루프가 아니라, 멈출 줄 아는 루프.

셀 수 있는 것은 기준으로, 셀 수 없는 것은 회차로

통과선으로 닫는다

- 검사기가 통과를 내면 끝
- 상한은 **최대** — 여기까지만 돈다

회차로 닫는다

- 정해진 횟수를 채우면 끝
- 3회는 **최소이자 최대**
- 회차마다 보는 렌즈를 바꾼다

"품질 8점 이상이면 통과"로 닫으면 글을 쓴 쪽이 자기 글에 점수를 주기에 재현성이 망가진다. 회차는 관측 가능하고 · 늘기만 하고 · 반드시 끝에 닿는다.

에이전트 = 모델 + 하네스

모델이 정하는 것

- 문장을 얼마나 잘 쓰는가
- 코드를 얼마나 잘 짜는가

하네스가 정하는 것

- 무엇을 손댈 수 있는가
- 무엇을 보고 끝났다고 하는가

하네스가 세우는 네 개의 벽

```
┌──────────────────────────────────────┐
│ 권한   손댈 수 있는 것과 없는 것을 가른다
│ 기억   다음 바퀴로 무엇이 넘어가는지 정한다
│ 관측   무엇을 보고 판단할지 고정한다
│ 완료   무엇이 되면 끝인지 선언한다
└──────────────────────────────────────┘
    루프는 이 안에서만 돈다
```

명세가 루프를 설계한다.

```
조건 7가지에 답한다
   ▼
명세가 완성된다 — 여기까지 코드는 생기지 않는다
   ▼
통과선으로 닫는 루프 — 검사기가 통과를 내면 끝
   ▼
회차로 닫는 루프 — 셀 수 없는 것은 정해진 횟수로
   ▼
결과물 하나
```

> 셀 수 있는 것은 전부 계약으로. 셀 수 없는 것은 회차로.

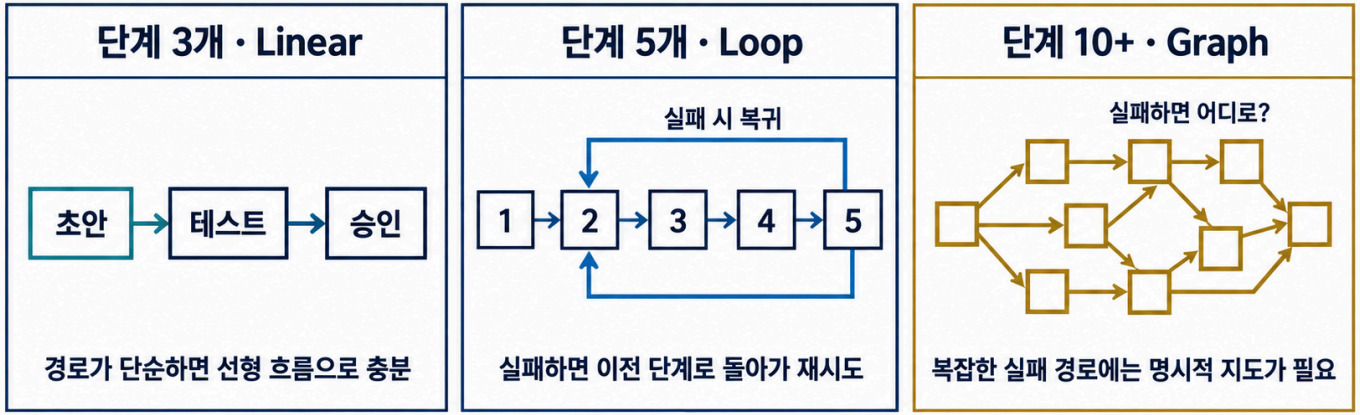
이 루프를 그래프로 확장한다. 작업이 노드가 되고, 실패 라우팅이 간선(edge)이 된다.

잘 도는 루프도 못 하는 네 가지

하고 싶은 일	루프에서는
돌리기 전에 구조를 눈으로 검사	코드를 읽어야만 안다
멈춘 자리에서 다시 시작	프로세스가 끝나면 처음부터
실패한 갈림길부터 다시 시도	바퀴 전체를 다시
뒤 단계에서 앞 단계로 되돌아가기	되돌아갈 길이 없다

> 넷 다 **모델을 바꿔서 해결되는 문제가 아니다.** 실행 순서가 코드 안에 박혀 있어서 생기는 일이다.

언제 그래프가 필요해지는가?



단순 루프가 감당하기 어려운 복잡한 분기부터 Graph가 필요해진다.

실패 원인이 여럿

원인마다 분기 경로가 다르면, 원인 수만큼 갈림길 파생

나눠서 동시에

한 줄로 순차 실행은 느리다. 나누면 **합치는 규칙**이 필요.

넘길 기록이 생김

단계 사이에 무엇을 넘길지 정해야 한다. 그것이 **상태(state)**다.

단계가 늘어나면 순서를 코드 안에 둘수록 블랙박스가 증가한다.

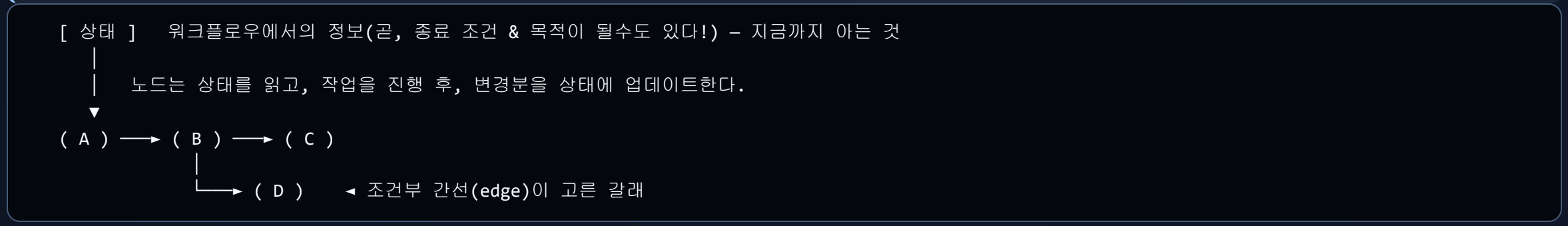
문제정의에서 출발하는 그래프 아키텍처 워크플로우

무엇을?	어떤 질문?
상태 · 노드 · 간선	무엇을 기억하고, 누가 일하고, 순서를 누가 정하나
조건부 간선	갈림길을 무엇을 보고 고르나
병렬과 합침	동시에 쓴 결과를 어떻게 합치나
체크포인트와 사람	어디서 멈추고, 누가 방향을 고르나
루프와 그래프 비교	무엇이 좋아지고, 무엇을 치르나

> 핵심은 **성능이 아니라 구조**다. 오후 실습에서는 같은 주제를 — 한 번은 루프로, 한 번은 그래프로 진행.

그래프 — 일의 순서를 코드 밖으로 꺼낸 것

> **할 일**은 node(노드),
> **다음에 무엇을**은 edge(간선),
> **지금까지 아는 것**은 state(상태).



이 세 낱말을 이름 그대로 쓰는 구현체가 있습니다 — 예: LangGraph.

상태(State) — 그래프 전체가 공유하는 작업 기억

노드가 끝날 때마다 **통째로 저장**. 무거우면 노드 수만큼 곱해진다.

```
좋은 상태 — 경로와 판정만 담는다
  stage_verdict : "SHORT"
  missing       : ["image"]
  page_path     : "output/site.html"
  round         : 2

나쁜 상태 — 본문을 통째로 담는다
  raw_html      : "<html> … 30만 자"
  everything    : { ... }
  result1, result2, temp
```

판정 값은 **반드시** 상태에 남기고 전달한다.— edge는 이를 읽어서 판단을 진행한다.

노드(Node) — 상태를 읽고 변경분을 돌려주는 한 가지 책임

노드가 될 수 있는 것

- 함수 · 파일 읽기 · 웹 검색
- 검사 스크립트 실행
- 사람의 승인 대기

노드의 규칙

- 한 노드는 한 가지 책임만
- 다른 노드를 직접 부르지 않는다
- 바뀐 부분만 돌려준다

노드는 반드시 에이전트가 필요하지 않다. 특히, 판정은 LLM에 맡기지 않는다.

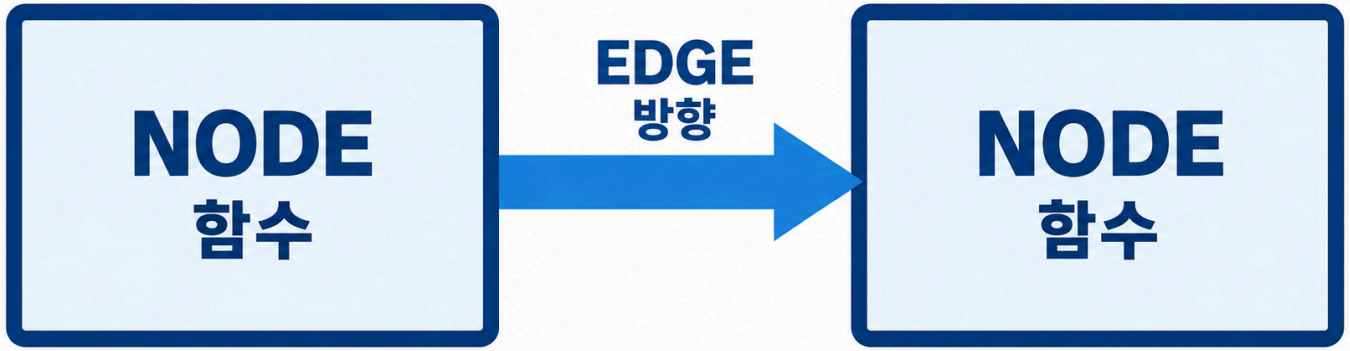
간선(Edge) — 다음 실행 위치를 잇는 제어선

> 노드는 **일을 하고**, 간선은 **다음에 무엇을 할지** 정한다. 워크플로우를 이 둘을 가지고 설계하는게 그래프의 거의 전부이다.

	정적 간선(여러개면 병렬)	조건부 간선
하는 일	무조건 다음으로	상태를 보고 고른다
정해지는 때	그릴 때	돌 때
쓰는 자리	순서가 고정된 곳	갈림길

워크플로우 순서가 코드 안이 아니라 edge 목록에 있다.

본질은 결국 함수들의 연결이다



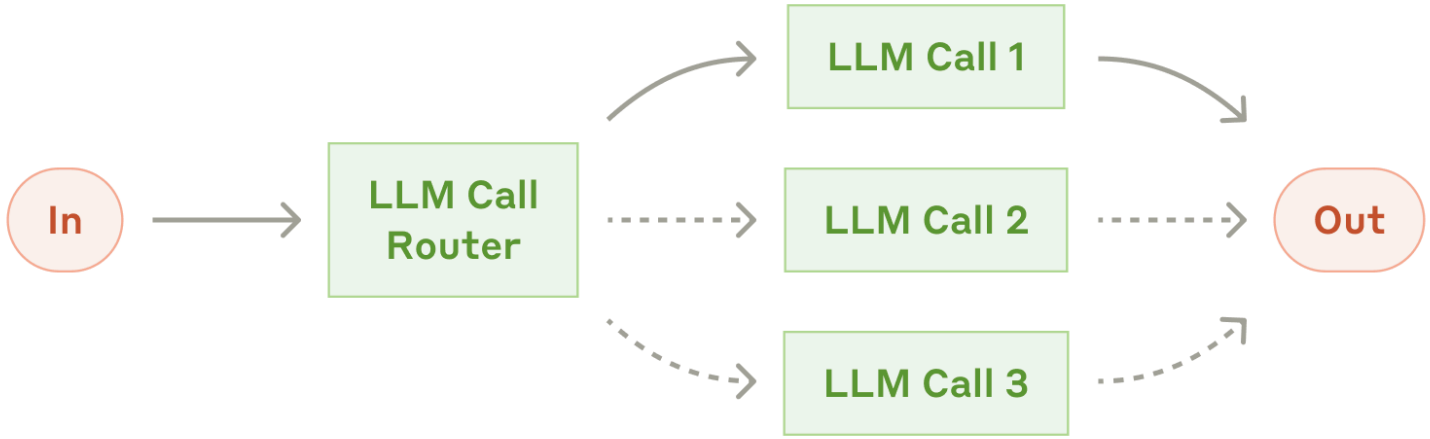
노드(Node): 현재 상태를 받아 계산하고 갱신하는 함수

엣지(Edge): 다음에 실행할 노드를 정하는 경로

```
1  상태를 읽는다
2  노드가 돈다
3  노드가 변경분을 돌려준다
4  합침 규칙(reducer)이 상태에 반영한다
5  라우터가 그 결과를 읽는다
6  다음 노드를 고른다
7  1로 돌아간다
```

> 루프의 한 바퀴(행동 → 관측 → 판단 → 반복)와 같은 모양. 달라진 것은 **3·4·5가 서로 다른 부품의 일로 떨어져 있다는 것**뿐.

핵심 아키텍처 1 : 조건부 간선 — 상태를 읽고 경로를 고른다



라우터가 하는 일은 하나다. — **상태를 읽고 갈래 이름 하나를 돌려준다.** 파일을 쓰지도, 검사기를 돌리지도, 상태를 바꾸지도 않는다.

출처: Anthropic, "Building Effective Agents" · anthropic.com

워크플로우 상태전이는 두 종류다.

```
데이터 상태전이 — 값이 바뀐다 · 노드가 만든다
   verdict : UNKNOWN  ──►  SHORT

제어 상태전이 — 실행 위치가 바뀐다 · 간선이 만든다
   check_stage  ──►  research
```

> 루프에서는 블랙박스에 해당하는 개념이다. 검사기를 돌리고 → 종료 코드를 보고 → 다음 작업.

그래프는 이 둘을 설계할 수 있다. 노드는 값을, edge는 경로를 바꾼다.

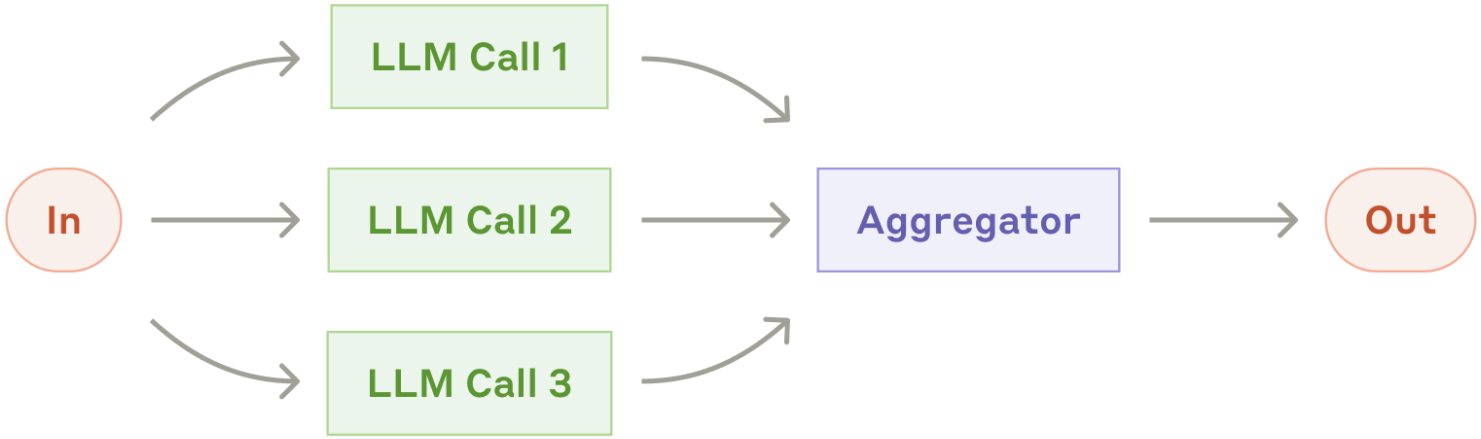
검사기(test)는 핵심 node이다.

라우터는 스크립트를 돌릴 수 없다. 판정 값은 항상 **상태에 있고** edge는 state만 본다.

	루프에서 검사기는	**그래프에서 검사기는**
정체	돌려 보고 종료 코드를 읽는 것	판정 값을 상태에 쓰는 **노드**
자리	작업과 작업 사이	그래프 안의 한 칸

당연히 작업이 8개라고 노드가 8개가 되진 않는다. 검사기와 사람이 서는 자리 둘이 올라오면 **8+개**가 된다.

핵심 아키텍처 2 - 정적 edge 두 개 : 병렬



edge를 둘 걸면 **한 step에 둘 다 실행.** 하나를 고르려면 조건부 edge를 써야한다.

출처: Anthropic, "Building Effective Agents" · anthropic.com

합침 규칙(reducer) — 동시에 쓴 결과를 어떻게 합치나

> 합침 규칙 없이 두 갈래가 같은 칸에 쓰면 **그래프가 멈춘다.** 조용히 사라지는 것이 아니라 오류를 내고 멈춘다.

```
이어 붙이기가 맞는 칸
   기록   노드마다 한 줄씩 쌓인다 — 겹칠 일이 없다

이어 붙이기가 틀린 칸
   목록   재조사가 같은 항목을 다시 돌려준다
          이어 붙이면 3개가 6개가 된다 → 이름으로 합친다
```

병렬은 속도의 문제가 아니라 합침 규칙의 문제

LangGraph는 한 번에 노드 하나씩 실행하는 게 아니고, 걸음(superstep) 단위로 돈다.

```
한 걸음 =
   1. 지금 "방아쇠가 당겨진" 노드들을 전부 동시에 실행한다
   2. 각 노드가 낸 변경분을 모아서 상태(state)에 반영한다 (reducer가 여기서 일한다)
   3. 끝난 노드들의 간선(edge)을 보고, 다음 걸음에 누구를 켤지 방아쇠를 당긴다
```

체크포인트 — 멈춘 자리에서 다시 시작

루프에서 되물음

- 화면에 질문이 뜬다
- 그 자리에서 답한다
- 프로세스는 살아 있다

그래프에서 멈춤

- 상태가 저장된다
- 그래프가 **끝난다**
- 답하면 그 자리에서 재개된다

멈춤 지점 앞은 다시 돌아도 같은 결과여야 한다. 재개하면 그 함수가 첫 줄부터 다시 돈다.

사람이 고르는 것은 '경로'

게이트	사람이 보는 것	고를 수 있는 길
정보 단계	못 채운 칸의 목록	더 모은다 · 부족한 채로 진행 · 멈춘다
결과 단계	만들어진 결과물	앞 단계로 되돌아간다 · 다시 만든다 · 확정한다

> 없는 정보는 몇 바퀴를 더 돌아도 없다. 부족한 채로 넘어가되, **못 채운 칸은 보고서에 그대로 적는다.**

결과 단계에서 정보 단계로 되돌아가기 — 루프에서는 불가능했고, 그래프에서는 edge 하나로 설계 가능

내부 루프는 에이전트가, 외부 루프는 사람이 쥔다.

```
기계가 닫는 순환 — 상한까지 스스로 돈다
   검사 ⟶ 부족 ⟶ 다시 모은다 ⟶ 검사
   만들기 ⟶ 검사 ⟶ 고친다 ⟶ 검사

사람이 닫는 순환 — 그래프가 멈추고 방향을 묻는다
   게이트 ⟶ 어디로 갈지 고른다 ⟶ 그 자리에서 재개
```

> 기계는 **셀 수 있는 것**을 닫고, 사람은 **방향**을 정한다. 바깥 루프까지 기계에 넘기면, 무엇이 왜 그렇게 되었는지 아무도 모르게 된다.

같은 일을 두 구조로 짜면

축	루프	그래프
실행 순서	코드에 박혀 있다	간선으로 선언한다
분기	`if` 문 안에 숨는다	조건부 간선 = 데이터
상태	그냥 공유 변수	명시적으로 선언한다
병렬	직접 만든다	간선 구조가 곧 병렬
실패 후 재실행	바퀴 전체	체크포인트부터
되돌아가기	앞 단계로 못 간다	간선 하나
구조 확인	코드를 읽어야 안다	돌리기 전에 그림으로
코드 길이	짧다	단계가 적으면 오히려 길다

> 계약은 **그대로.** 바뀌는 것은 실행 구조뿐. 계약을 그래프용으로 "개선"하는 순간 두 판을 같은 표에 올릴 수 없다.

얻는 것 셋, 치르는 대가 셋

얻는다

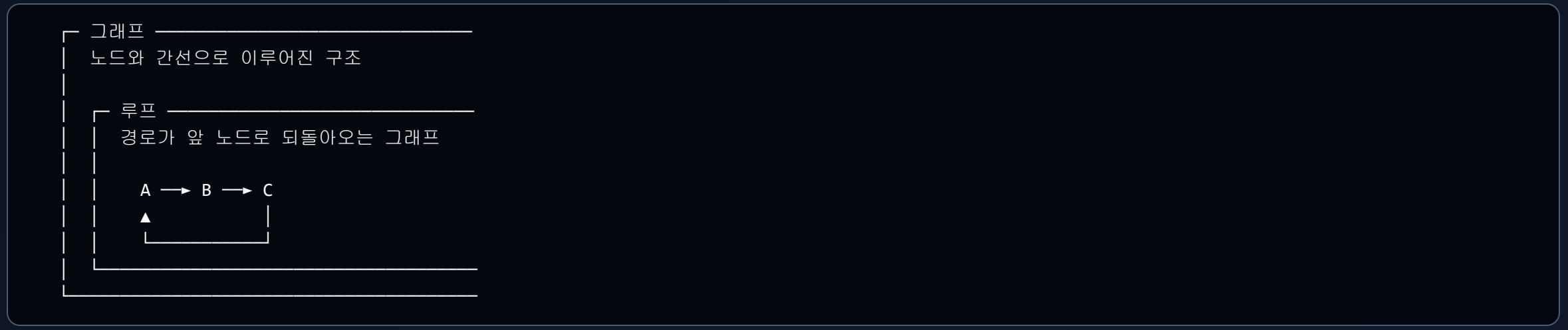
- **병렬** — 나눌 수 있는 일을 동시에
- **부분 재실행** — 실패한 자리만 다시
- **가시성** — 구조가 그림으로 보인다

치른다

- 상태를 **명시적으로** 선언해야 한다
- 노드가 서로를 **몰라야** 한다
- 분기 조건을 코드에서 빼내야 한다

이 셋이 필요 없으면 **루프가 낫다** — 그래프가 오히려 코드를 길게 만들고, 의존성 문제가 커지며, 직관적 시각화를 위해 외부 툴이 필요하다.

루프는 그래프의 한 모양



세대 교체가 아니라 **포함 관계**, 하나의 가닥에 가깝다. 루프가 낡은 것이 아니라, 필요할 때 그래프를 설계한다.

그래프가 개선하지 못하는 것

> 없는 정보를 만들어 내지 못한다. **못 채운 칸은 루프로 짜든 그래프로 짜든 똑같이 못 채운 칸이다.**

무엇이	좋아지나
실행 시간	나눌 수 있는 일이 있을 때만
복구 능력	실패한 자리부터 다시 — 확실히 좋아진다
정성적 완성도	**좋아지지 않는다** — 앞 회차가 다음 회차의 입력이다
사람의 판단	대신해 주지 않는다

그래프가 바꾸는 것은 블랙박스 개선과 실행 효율 및 워크플로우 재현성 & 복구 능력 등이지 결과의 존재 여부가 아니다.

루프인가 그래프인가

묻는 것	그렇다면
나눠서 동시에 돌릴 단계가 있는가	그래프
실패한 자리부터 다시 시작해야 하는가	그래프
돌리기 전에 구조를 눈으로 봐야 하는가	그래프

> 셋 다 아니면 **루프가 낫다.** 옮길 때는 **계약은 그대로 두고 실행 구조만**

가장 큰 그래프가 목표가 아니라, 일의 품질을 올리는 가장 작은 구조가 목표.

팩토리 : 경계의 확장

무엇	예전	지금
분량이 모자란가 · 같은 말을 복사했는가	사람	코드
링크가 살아 있는가	아무도 안 봄	코드
이것이 내가 원하던 것인가	사람	**사람**

> AI가 할 수 있는 것은 AI에게, 사람의 시간은 사람이 아니면 못 보는 것에만

병목은 생성이 아니라 검증이다. 자율성은 우리의 인지 안에서 검증할 수 있는 만큼이 한계다.